

# Chunking Strategy Evaluation: Detailed Technical Report

The first step for a complete RAG project — find the best chunking strategy for your document domain before tuning anything downstream.

| Field              | Value                                                    |
|--------------------|----------------------------------------------------------|
| Step               | First step of a complete RAG project: chunking           |
| Status             | Complete — 2 runs finished                               |
| Document domain    | Annual reports (P&G Annual Report 2025)                  |
| Golden dataset     | 50 questions (data/pg_2025_gold_evaluation_dataset.json) |
| Run 1              | Substring-match evaluation (2026-09-10)                  |
| Run 2              | Substring + semantic-similarity evaluation               |
| Winner (semantic)  | recursive — semantic Recall@3 = 0.58                     |
| Winner (substring) | fixed_size — Recall@3 = 0.38                             |
| Best value         | recursive — top quality at 2nd-lowest cost (\$0.0035)    |

## Table of Contents

- 1. [Objective & Scope](#)
- 2. [Document Domain](#)
- 3. [Golden Dataset](#)
- 4. [Chunk Data Structure](#)
- 5. [Chunking Strategies](#)
- 6. [Azure Services & Technical Aspects](#)
- 7. [Experiment Pipeline](#)
- 8. [Evaluation Methodology](#)
- 9. [Results — Run 1 \(Substring Match\)](#)
- 10. [Results — Run 2 \(Substring + Semantic\)](#)
- 11. [Comparative Analysis — Run 1 vs Run 2](#)
- 12. [Cost & Latency Analysis](#)
- 13. [Recommendation](#)
- 14. [Findings & Limitations](#)
- 15. [Reproducibility & References](#)

## 1. Objective & Scope

**Goal:** Determine which chunking strategy produces the best retrieval quality for annual-report documents, so that every later step of the RAG project (embeddings → retrieval → generation → optimization) builds on the right foundation.

**Why chunking first:** Chunking is the earliest irreversible decision in a RAG pipeline. A bad split permanently destroys context (e.g. a table row separated from its header) that no embedding model or reranker can recover. All later experiments therefore fix this step's winner and vary only one factor at a time.

**Why annual reports:** dense financial tables where numbers differ only by column position (84,284 vs 84,039), mixed prose/statements/footnotes, and questions spanning single-cell lookup to multi-step reasoning.

**Scope:** This step evaluates retrieval only (no LLM generation). Retrieval = embed 50 golden questions → top-5 vector search → score against ground truth. Generation quality (faithfulness, relevancy) is evaluated in a later step.

---

## 2. Document Domain

| Attribute      | Value                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Source         | data/raw_pdf/P&G_AnnualReport.pdf (Procter & Gamble, FY2025) — 4.28 MB, 301 pages                                                   |
| Processed form | data/markdown/P&G_AnnualReport.md (0.36 MB) via pymupdf4llm (src/utils/pdf_to_markdown.py)                                          |
| Loader         | src/utils/document_loader.py → load_all_documents(use_markdown=True) prefers markdown, splits on headings into "page-like" sections |

---

## 3. Golden Dataset

**Location:** data/pg\_2025\_gold\_evaluation\_dataset.json · **Builder:** buildgolddataset.py (curated list → JSON) · **Size:** 50 entries (q001–q050)

### 3.1 Schema (8 fields per entry)

| Field                      | Description                                                                            |
|----------------------------|----------------------------------------------------------------------------------------|
| id / question              | Question ID / natural-language question asked to the retriever                         |
| ground_truth_answer        | Human-verified answer (reserved for generation evaluation in a later step)             |
| supporting_text            | Evidence span the retrieved chunk must contain / resemble — the retrieval ground truth |
| source / source_span       | {document, page, section} provenance + {page, start_text, end_text} span               |
| question_type / difficulty | One of 13 types (§3.2) / easy / medium / hard                                          |

```
{
  "id": "q001",
  "question": "What were P&G's net sales in fiscal 2025?",
  "ground_truth_answer": "$84.3 billion",
  "supporting_text": "Net sales $ 84,284 $ 84,039 — %",
  "source": {"document": "P&G_AnnualReport.pdf", "page": 19, "section": "Summary of 2025 Results"},
  "source_span": {"page": 19, "start_text": "Net sales", "end_text": "$ 84,284 $ 84,039 — %"},
  "question_type": "fact",
  "difficulty": "easy"
}
```

### 3.2 Question-type taxonomy (measured from the dataset file)

| Type               | n    | Tests                                                                                                  | Sample                                      |
|--------------------|------|--------------------------------------------------------------------------------------------------------|---------------------------------------------|
| fact               | 19   | Single-figure lookup                                                                                   | P&G's net sales in FY2025?                  |
| cause_effect       | 6    | Reason behind a result                                                                                 | Why did operating income rise 10%?          |
| context            | 5    | Synthesized narrative                                                                                  | Major challenges in FY2025?                 |
| table_lookup       | 5    | Cell-level extraction                                                                                  | Beauty's net sales?                         |
| comparison         | 4    | Cross-segment contrast                                                                                 | North America vs European Focus markets?    |
| multi_sentence     | 2    | Answer spans sentences                                                                                 | Cash returned to shareholders?              |
| list               | 2    | Enumerated sets                                                                                        | Daily-use categories in focused portfolio?  |
| boundary_sensitive | 2    | Exact span edges                                                                                       | Why Fabric & Home Care sales flat?          |
| 5 singletons       | 1 ea | definition_and_fact, multi_concept, multi_factor, structured_data, definition_and_multi_step_reasoning | e.g. factors behind \$14.6B free cash flow? |

Stats: difficulty 24 easy / 14 medium / 12 hard; avg lengths — question 65, evidence 106, answer 63 chars; all questions draw from 6 PDF pages (19, 22–24, 43–44): a *precision* test, not a coverage test.

## 4. Chunk Data Structure

All strategies emit the same dataclass (src/chunking/base.py), so embedding → indexing → evaluation is strategy-agnostic:

```
@dataclass
class Chunk:
    id: str      # fixed_ / rec_ / sem_ / md_ / pc_ + source_page_index
    text: str
    source: str
    page: int    # heading ordinal, NOT pdf page (see §2)
    section: str = "" # heading text (populated by markdown_aware only)
```

```
chunk_index: int = 0
metadata: dict = field(default_factory=dict)
```

Index IDs are sanitized (re.sub(r'^a-zA-Z0-9\_=-', '\_', id)) for Azure AI Search key constraints. Token counting uses tiktoken cl100k\_base everywhere (chunk stats + cost accounting). parent\_child stores {parent\_text, parent\_index} in metadata so the full parent can be returned as LLM context at generation time.

## 5. Chunking Strategies

Same input (markdown sections), same downstream pipeline — only the splitter differs.

**fixed\_size** (fixed\_size.py): pure token-window slicing — encode with tiktoken, slide a 512-token window with 50-token overlap. Deterministic, O(n), no external calls; cuts mid-sentence/mid-table. → 405 chunks, avg 225.0 tokens.

**recursive** (recursive.py): LangChain RecursiveCharacterTextSplitter, separators ["\n\n", "\n", ". ", " ", ""], chunk\_size=512, overlap=50 — but length\_function=len counts **characters, not tokens**, so "512" ≈ ~128 tokens of English. Same nominal config as fixed\_size, very different granularity. → 1,269 chunks, avg 68.4 tokens.

**semantic** (semantic.py): LangChain SemanticChunker over AzureOpenAIEmbeddings, percentile breakpoint 75.0; splits where inter-sentence similarity drops. Requires embedding calls at chunk time — measured **126 s** chunking vs <1 s for all others. → 742 chunks, avg 115.5 tokens.

**markdown\_aware** (markdown\_aware.py): custom regex on headings ^#{1,6}\s+(.+)\$. one chunk per section, tables kept whole, oversize sections (>1024 chars) sub-split on paragraphs. Fastest chunker (12 ms), uneven sizes. → 602 chunks, avg 142.6 tokens.

**parent\_child** (parent\_child.py): two-level split — parents 1024 chars / overlap 0 → children 256 chars / overlap 30. Child text is embedded for retrieval; parent rides along in metadata. Highest-resolution index. → 2,179 chunks, avg 41.4 tokens.

| Strategy       | Unit      | Size / overlap        | Chunks | Avg tokens |
|----------------|-----------|-----------------------|--------|------------|
| fixed_size     | tokens    | 512 / 50              | 405    | 225.0      |
| recursive      | chars     | 512 / 50              | 1,269  | 68.4       |
| semantic       | sentences | p75 breakpoint        | 742    | 115.5      |
| markdown_aware | chars     | 1024 max              | 602    | 142.6      |
| parent_child   | chars     | 1024 / 256, ov 0 / 30 | 2,179  | 41.4       |

## 6. Azure Services & Technical Aspects

| Service         | Setup                                                                                                                                                                  | Role in this step                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Azure OpenAI    | rag-evaluation-optimization.cognitiveservices.azure.com, API 2023-05-15, text-embedding-3-small (1536-d, batch 100), cosine via numpy (src/embeddings/azure_openai.py) | chunk, query, and eval embeddings; gpt-4o provisioned for generation later |
| Azure AI Search | rag-experimentation-vector-db.search.windows.net, index rag-evaluation-index, HNSW (default profile)                                                                   | vector index + top-5 retrieval (src/retrieval/azure_ai_search.py)          |

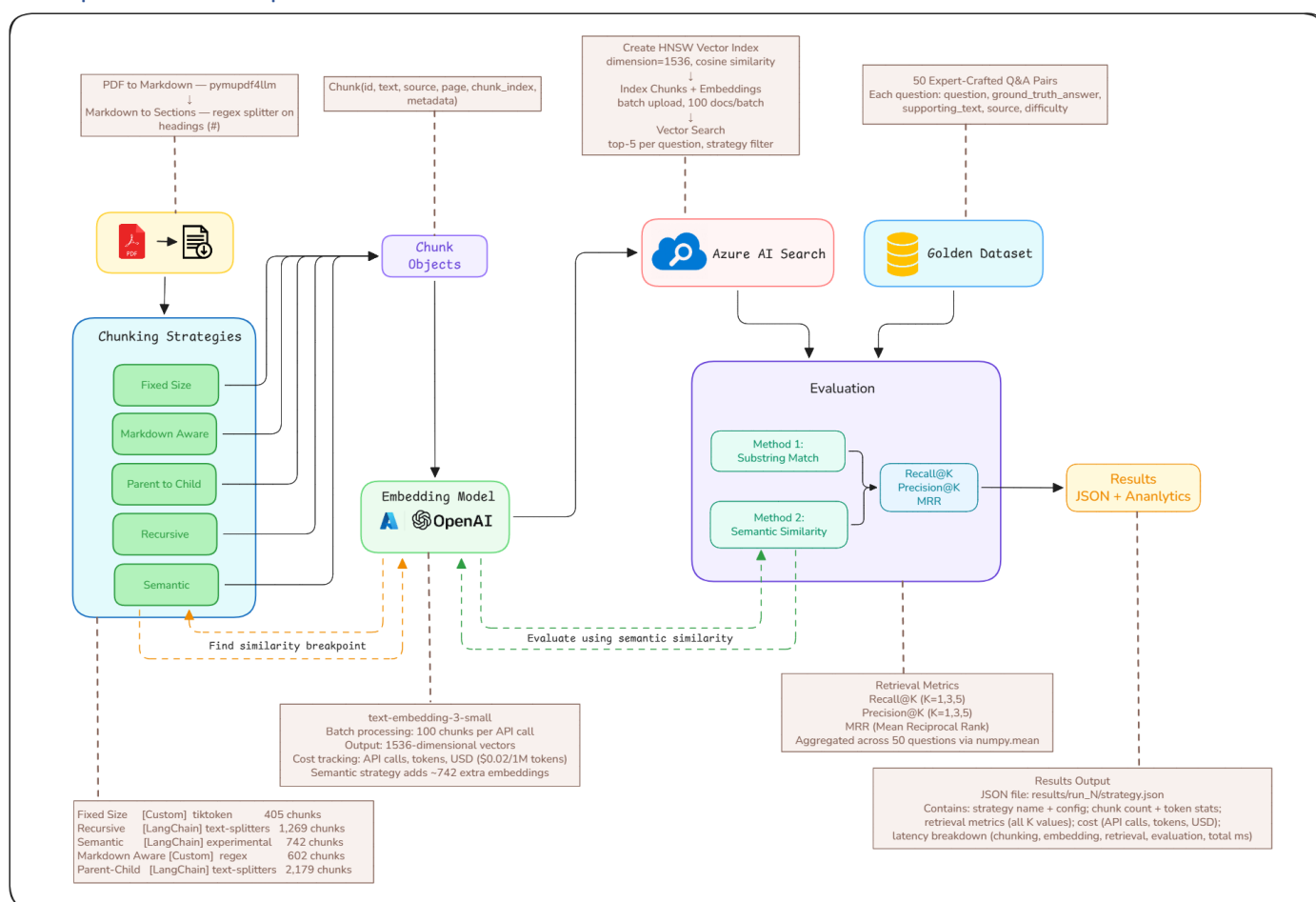
Index fields: id (key) · text (searchable) · source, page, strategy (filterable — each run isolated via strategy eq '<name>\_v2') · section (searchable) · embedding (1536-d vector). Retrieval is pure vector (search\_text=None, VectorizedQuery k=5); uploads in batches of 100; each script deletes + recreates the index, then cleans up. bm25\_search() / hybrid\_search() exist for a later retrieval-comparison step.

Portal telemetry (Run 1 screenshots): AOAI 1.03M input tokens, 530 requests, 269.78 ms avg latency; Search 1.26 queries/s, 13.85 ms latency, 6 MB vector storage.

Pricing applied: embeddings **\$0.02 / 1M tokens** (CostTracker: tokens/1e6 × 0.02). Reference: embedding-large (3072-d) ≈ \$0.13/1M; GPT-4o \$2.50 in / \$10.00 out; mini \$0.15 / \$0.60.

Shared constants: tokenizer tiktoken cl100k\_base · top-k = 5 (RETRIEVAL\_TOP\_K) · metric cutoffs k = 1/3/5 · semantic threshold cosine ≥ **0.6**.

## 7. Experiment Pipeline



## 8. Evaluation Methodology

**Substring (Run 1, retrieval\_metrics.py):** supporting\_text.lower() in chunk.text.lower() — binary, no partial credit. Weaknesses: markdown artifacts break matches, paraphrases score 0, 90% overlap = 0%.

**Semantic (Run 2, semantic\_relevance.py):** embed evidence + retrieved chunks with text-embedding-3-small; relevant iff cosine  $\geq 0.6$ . Supports pre-embedded once (1 batch); each retrieved chunk embedded individually (~650 calls/strategy — the reason Run 2 costs more). avg\_similarity (mean cosine over top-5) adds a threshold-free graded signal.

| Metric      | Rule                                                          |
|-------------|---------------------------------------------------------------|
| Recall@k    | 1.0 if any top-k chunk relevant, else 0 (averaged over 50 Qs) |
| Precision@k | fraction of top-k relevant (averaged)                         |
| MRR         | mean of 1/rank of first relevant hit (0 if none)              |
| semantic_*  | same rules with cosine $\geq 0.6$ as relevance                |

Worked example — "What were P&G's net sales in fiscal 2025?": a 225-token fixed\_size chunk containing the table row → substring HIT. A 41-token parent\_child chunk reading "Net sales increased \$245M to \$84.3B..." → substring MISS, cosine  $\approx 0.8$  → semantic HIT. This one mechanism explains the ranking flip in §11.

---

## 9. Results — Run 1 (Substring Match)

| Strategy       | R@1  | R@3         | R@5  | P@3   | MRR          | Cost     | Time  |
|----------------|------|-------------|------|-------|--------------|----------|-------|
| fixed_size     | 0.22 | <b>0.38</b> | 0.38 | 0.127 | <b>0.293</b> | \$0.0018 | 235 s |
| markdown_aware | 0.20 | 0.34        | 0.36 | 0.113 | 0.264        | \$0.0017 | 190 s |
| semantic       | 0.12 | 0.28        | 0.30 | 0.093 | 0.194        | \$0.0017 | 300 s |
| recursive      | 0.14 | 0.26        | 0.34 | 0.087 | 0.216        | \$0.0017 | 214 s |
| parent_child   | 0.14 | 0.22        | 0.22 | 0.073 | 0.180        | \$0.0018 | 240 s |

Read: Recall@3 strictly decreases with chunk size (225→38%, 143→34%, 116→28%, 68→26%, 41→22%) — big chunks are likelier to contain the exact evidence string; small chunks split evidence (avg 106, up to 284 chars) across boundaries. Indexing  $\approx 86$ –91K tokens per strategy → cost flat ( $\sim$ \$0.0017). Retrieval ( $\sim$ 3.1 s/query) dominates runtime.

---

## 10. Results — Run 2 (Substring + Semantic)

Substring columns reproduce Run 1 exactly — the pipeline is deterministic; Run 2 is a strict superset. Semantic results (threshold 0.6):

| Strategy       | Sem R@1     | Sem R@3     | Sem R@5     | Sem P@3       | Sem MRR       | Avg sim       |
|----------------|-------------|-------------|-------------|---------------|---------------|---------------|
| recursive      | 0.38        | <b>0.58</b> | <b>0.64</b> | 0.3400        | <b>0.4873</b> | 0.5488        |
| parent_child   | 0.36        | 0.56        | 0.62        | 0.3400        | 0.4673        | <b>0.5502</b> |
| semantic       | 0.36        | 0.56        | 0.58        | <b>0.3467</b> | 0.4550        | 0.5315        |
| markdown_aware | <b>0.42</b> | 0.52        | 0.58        | 0.3067        | 0.4753        | 0.5341        |
| fixed_size     | 0.26        | 0.42        | 0.44        | 0.2333        | 0.3350        | 0.5071        |

Cost (indexing + eval embeddings):

| Strategy       | API calls | Tokens  | Cost            | / question |
|----------------|-----------|---------|-----------------|------------|
| parent_child   | 650       | 142,050 | <b>\$0.0028</b> | \$0.000057 |
| recursive      | 646       | 177,207 | \$0.0035        | \$0.000071 |
| markdown_aware | 655       | 276,347 | \$0.0055        | \$0.000111 |
| semantic       | 652       | 276,626 | \$0.0055        | \$0.000111 |
| fixed_size     | 678       | 416,977 | \$0.0083        | \$0.000167 |

Latency (total 1,015–1,343 s): evaluation sweeps 814–878 s (~70–80% of wall clock); retrieval flat ~155 s; embedding 13–33 s; chunking  $\approx 0$  except semantic (126 s) vs markdown\_aware (12 ms). Fastest: markdown\_aware (1,015 s). Slowest: parent\_child (1,343 s, +32%).

---

## 11. Comparative Analysis — Run 1 vs Run 2

| Strategy       | Substring R@3 | Semantic R@3 | $\Delta$ |
|----------------|---------------|--------------|----------|
| fixed_size     | <b>0.38</b>   | 0.42         | +0.04    |
| markdown_aware | 0.34          | 0.52         | +0.18    |
| semantic       | 0.28          | 0.56         | +0.28    |
| recursive      | 0.26          | <b>0.58</b>  | +0.32    |
| parent_child   | 0.22          | 0.56         | +0.34    |

**The winner flips.** fixed\_size gains almost nothing (+0.04) — its chunks already contained exact text. Fine-grained strategies gain +0.28 to +0.34: semantic scoring credits the *meaning* their small chunks preserved when wording/segmentation broke exact matches. Evidence (~25 tokens) sits perfectly in a 68-token recursive chunk (evidence + one context sentence = ideal cosine signal-to-noise) but drowns in a 225-token window (lowest avg\_sim, 0.5071). parent\_child's 41-token children are the purest units (top avg\_sim 0.5502) at the cost of 2,179 vectors.

MRR confirms it: recursive 0.216  $\rightarrow$  **0.487** (first hit at rank ~2 — production-usable); fixed\_size barely moves (0.283  $\rightarrow$  0.335). Precision nuance: the semantic chunker leads Sem P@3 (0.3467) — topic-coherent chunks put *more* relevant items in top-3, which matters at generation time when the whole top-k enters the prompt (fewer distractors  $\rightarrow$  fewer hallucinations).

---

## 12. Cost & Latency Analysis

**What the dollars measure:** embedding input tokens only — (a) one-time indexing (~86–91K tokens) plus, in Run 2, (b) 50 pre-embedded supports, (c) ~450 recall-sweep embeddings, (d) the MRR sweep, (e) 50 avg-similarity batches. Excludes Search ops, egress, compute. Run 2 costs 3–5× Run 1 purely from (b)–(e): an **evaluation artifact**, not an indexing property. Production pays (a) once + one query embedding per question.

**Scaling (eval-style, \$0.02/1M):** 1K questions ≈ parent\_child \$0.06 / recursive \$0.07 / fixed\_size \$0.17; 10K ≈ \$0.6 / \$0.7 / \$1.7. Conclusion: **never pick chunking on embedding cost alone** — even the priciest strategy is <\$2 per 10K questions. The real cost lever arrives at the generation step (GPT-4o \$2.50/\$10.00 vs mini \$0.15/\$0.60).

**Latency:** eval sweeps are the bottleneck (~800–880 s of sequential per-chunk calls; batching them is the obvious future optimization). Retrieval is strategy-invariant (~3.1 s/query: ~270 ms query embedding + ~14 ms HNSW lookup + overhead) — index size has no effect at this scale. Semantic chunking's 126 s build-time tax recurs on every re-index: a hidden operational cost beyond its \$0.0055 bill.

---

## 13. Recommendation

| Dimension                   | Winner                                                                       | Runner-up                       |
|-----------------------------|------------------------------------------------------------------------------|---------------------------------|
| Semantic recall / MRR       | <b>recursive</b> (0.58 / 0.4873)                                             | parent_child (0.56 / 0.4673)    |
| Substring recall            | fixed_size (0.38)                                                            | markdown_aware (0.34)           |
| Semantic precision@3        | semantic (0.3467)                                                            | recursive / parent_child (0.34) |
| Top-1 accuracy              | markdown_aware (0.42)                                                        | recursive (0.38)                |
| Cost / latency / simplicity | parent_child (\$0.0028) / markdown_aware (1,015 s) / fixed_size (405 chunks) | recursive (\$0.0035)            |

**Overall: recursive — the strategy later steps build on.** #1 semantic quality, 2nd-cheapest (1.25× cheapest: +\$0.0007 buys +2pp recall, +0.02 MRR over parent\_child), zero chunk-time infrastructure, deterministic re-indexing. Follow-up: sweep true sizes (256/512/1024 chars, overlaps 0/50/100), since effective granularity drove the result. Conditional picks: exact-match compliance → fixed\_size; cheapest pilot → parent\_child (+ parent expansion at generation time); re-index speed / tables → markdown\_aware; top-3 precision → semantic chunker.

---

## 14. Findings & Limitations

**Findings:** (1) The evaluator picks the winner — publishing one number misleads. (2) Small, structure-respecting chunks win on meaning (+16pp over fixed windows — the reverse of the substring ranking). (3) Substring eval systematically under-reports everything except fixed\_size. (4) markdown\_aware is the efficient middle: top-1 leader, fastest, cheap. (5) Semantic chunking doesn't earn its tax — mid-pack quality at 126 s + complexity; spend the embedding budget on evaluation, not splitting. (6) Retrieval latency is flat across strategies.

**Limitations:** single document, 6 pages — cross-domain generalization untested. n=50, so each Recall@3 point is one question (gaps <0.04 ≈ ties). One threshold (0.6) + one embedding model — sensitivity untested. No generation eval yet (retrieval ≠ answers). parent\_child's parent-expansion feature unexercised. Cost scope excludes Search ops and compute — relative comparison only, not production budgeting.

---

## 16. Reproducibility & References

```
pip install -r requirements.txt # openai, azure-search-documents, langchain*, pypdf, tiktoken, ...
python buildgolddataset.py      # (re)generate golden dataset
python experiments/phase1_chunking/run2_recursive.py # Run 2 for one strategy
# → experiments/phase1_chunking/results/run_2/<strategy>.json
```

.env:

| Variable                                                                        | Value                                              |
|---------------------------------------------------------------------------------|----------------------------------------------------|
| AZURE_OPENAI_API_KEY / AZURE_OPENAI_ENDPOINT                                    | Azure OpenAI access                                |
| AZURE_OPENAI_API_VERSION                                                        | 2023-05-15                                         |
| AZURE_OPENAI_EMBEDDING_DEPLOYMENT / AZURE_OPENAI_CHAT_DEPLOYMENT                | text-embedding-3-small / gpt-4o (generation later) |
| AZURE_AI_SEARCH_ENDPOINT / AZURE_AI_SEARCH_API_KEY / AZURE_AI_SEARCH_INDEX_NAME | Search access / rag-evaluation-index               |

Each script recreates the index, runs 50 queries, saves JSON, and cleans up its <strategy>\_v2 documents — safe to run sequentially.